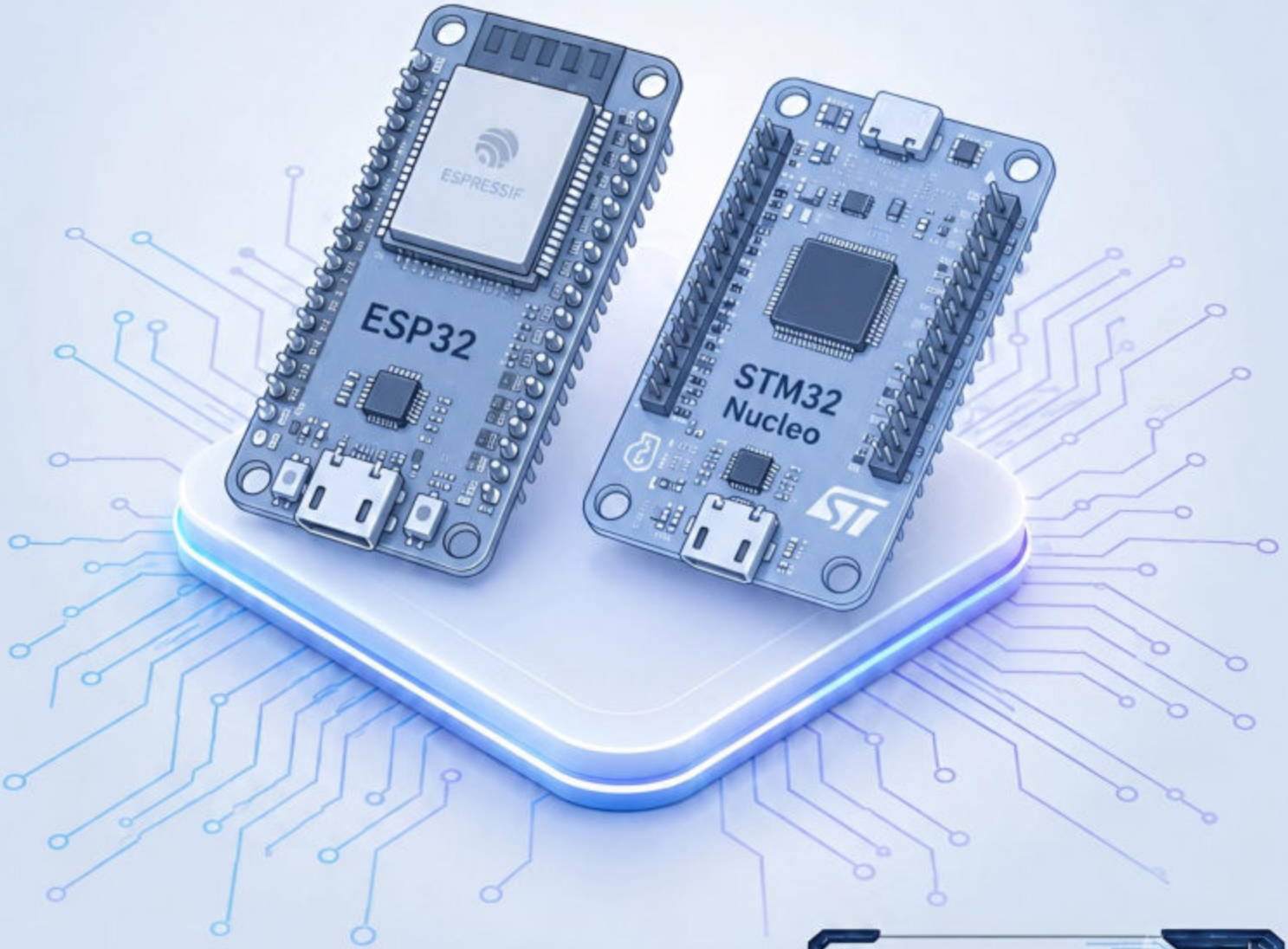
فريق نون التطوعي - السنة الرابعة

المعالجات الصغرية و نظمها

الملحق الثالث:

متحكمات ESP32 & STM32

قسم د.ماهر علاوي



كتابة وتدقيق: بلال السابق

تفريغ: سنا ذوالغنى - ليلاس إدلبي

قسم هندسة الالكترونيات والاتصالات

2026 - 2025

STM32 MICROCONTROLLER

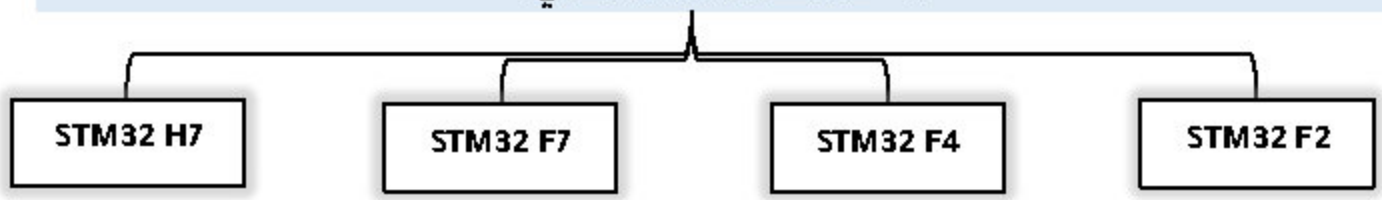
لمحة تاريخية

- ولدت شركة STM32 microcontroller عام 1987 نتيجة لدمج شركتين إحداهما فرنسية والأخرى إيطالية ..
- أطلقت الشركة عام 2007 أول منتجاتها بسلسلة STM32 F1 من عائلة المتحكمات STM32 التي تعتمد بنية ARM
- تمتاز عائلة المتحكمات STM32 باحتوائها على المعالج Cortex-M 32-bit الذي يتيح للمستخدم استخدام أدوات البرمجة من ARM
- تعد عائلة المتحكمات STM32 رخيصة السعر ، وقد تكون أرخص من بعض المتحكمات ذات البنية 8bit مثل AVR
- تتواجد معالجات ARM في الهواتف النقالة و الآلات الحاسبة و مشغلات الموسيقى و موجهات الشبكة Routers وغيرها الكثير من الأجهزة الالكترونية .

عائلة متحكمات STM32

تحتوي عائلة متحكمات STMR32 على 11 مجموعة مقسمة إلى ثلاثة أصناف :

1. متحكمات الأداء العالي :



تمتاز بقدرة حسابية عالية ، تنفذ التعليمات مباشرة من ذاكرة الفلاش ، سرعات تصل لـ400MHz ، مناسبة لتطبيقات الGraphics و معالجة الإشارة الرقمية .

2. متحكمات الأداء المتوسط :



تمتاز بأنها رخيصة الثمن و صغيرة الحجم ، وتمتاز بسرعات تصل لـ72MHz

3. المتحكمات ذات الاستهلاك القليل للطاقة :

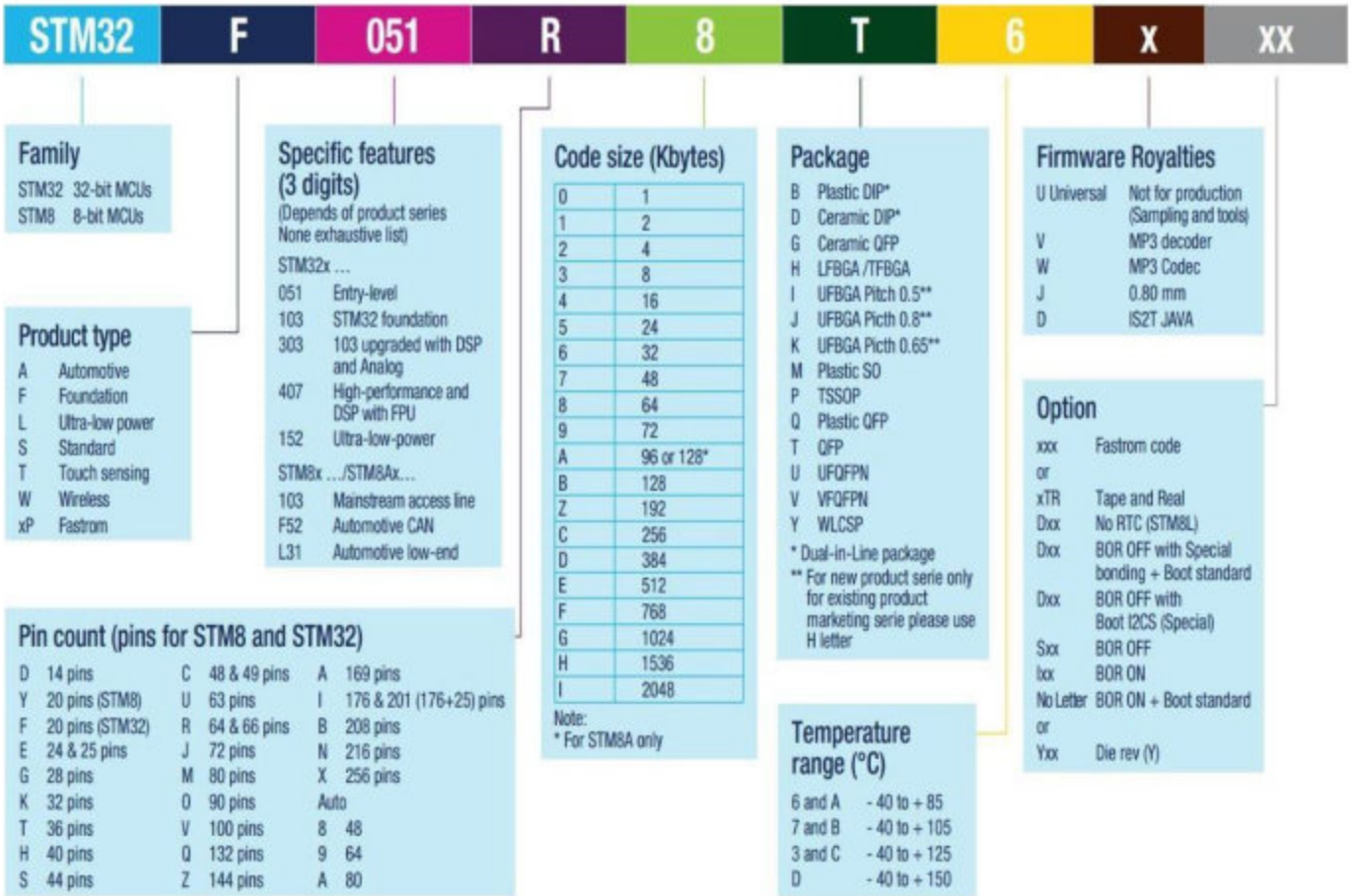
- STM32 L4+
- STM32 L4
- STM32 L1
- STM32 L0

تمتاز بأنها موفرة للطاقة وتدعم تقنية DVS التي تتيح تحسين استهلاك المعالج للطاقة بناءً على التردد المستخدم .

عائلات ARM وصولاً إلى ARM Cortex :

- ARM9 -
- ARM9E -
- ARM10 -
- ARM11 -
- ARM Cortex-A (A: Application) -
- ARM Cortex-R (R: Real time system) -
- ARM Cortex-M (M: eMbedded system) -

دلالة الاسم في متحكمات STM32



المتحكم STM32 F103C8T6 :

سندرس هذا المتحكم كمثال على متحكمات STM32،والذي يتمتع بالمواصفات الفنية التالية:

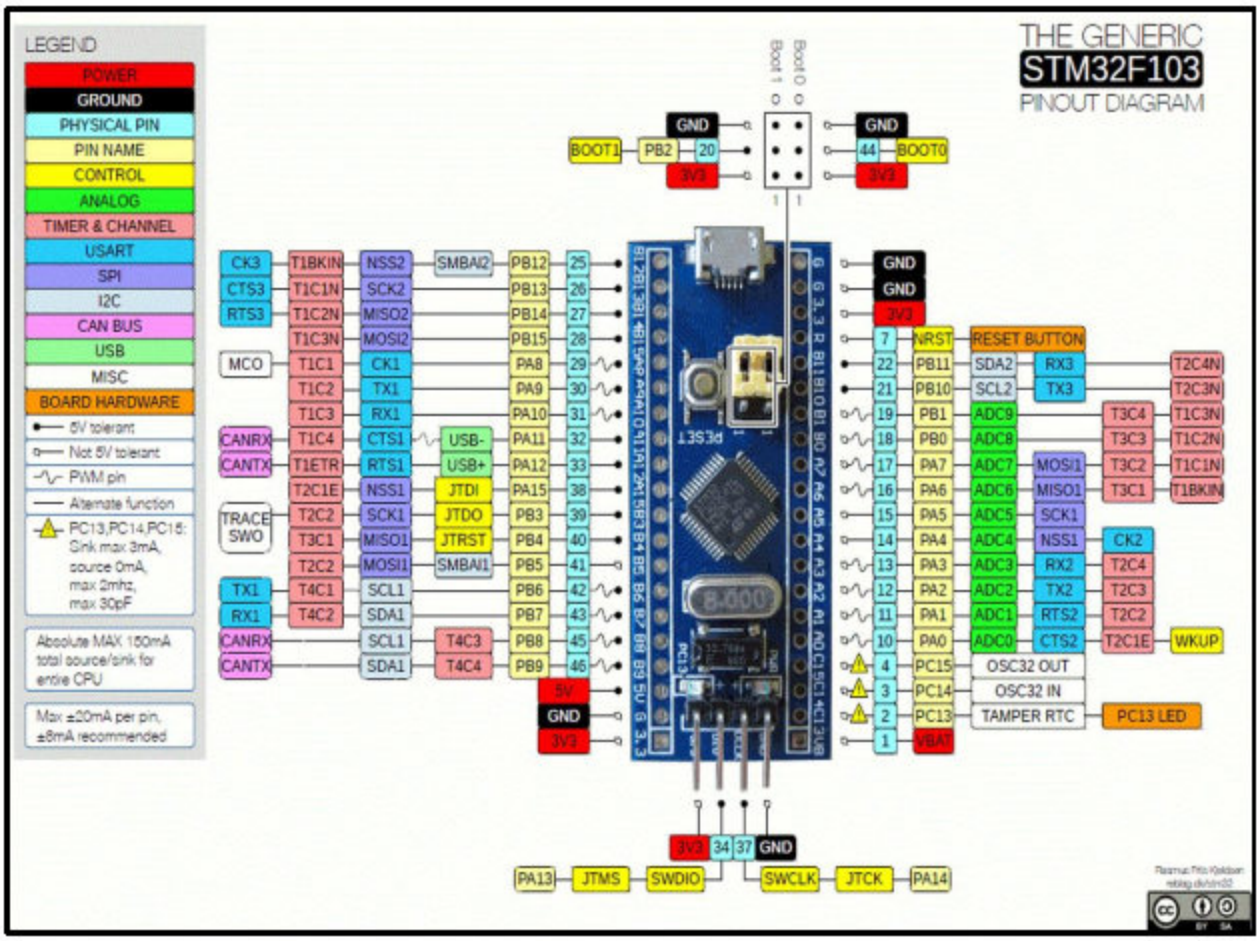
- Architecture : 32-bit ARM Cortex M3
- Operating Voltage : 2.7v to 3.6v
- CPU Frequency : 72 MHz
- Number of GPIO pins : 37
- Number of PWM pins : 12
- Analog input pins : 10 (12-bit)
- USART Peripherals : 3
- I2C Peripherals : 2
- SPI Peripherals : 2
- Can 2.0 Peripherals : 1
- Timers : 3 (16-bit) , 1 (PWM)
- Flash Memory : 64 KB
- RAM : 20 KB



مقارنة بين متحكم *Arduino UNO* و *STM32 F103C8T6* :

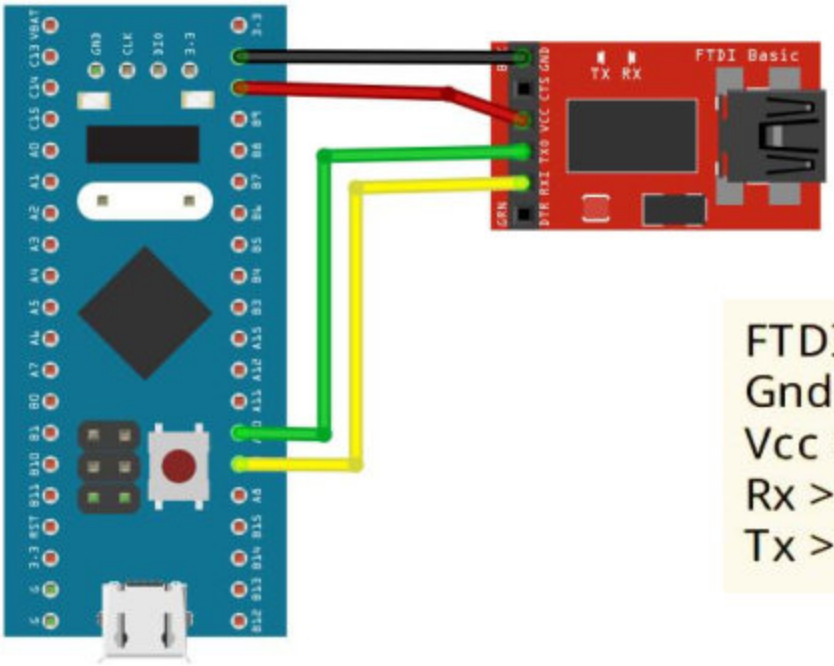
SPECS/BOARD	STM32F103C	ARDUINO UNO
Number of Cores	1	1
Architecture	32 Bit	8 Bit
CPU Frequency	72 MHz	16 MHz
WiFi	NO	NO
BLUETOOTH	NO	NO
RAM	20 KB	2 KB
FLASH	64 KB	32 KB
GPIO PINS	37	14
Busses	SPI, I2C, UART, CAN	SPI, I2C, UART
ADC Pins	10	6
DAC Pins	0	0

شكل المتحكم STM32 F103C8T6 :



تطبيقات على المتحكم STM32 F103C8T6 انطلاقاً من بيئة Arduino IDE :

أولاً يجب أن نعلم أن لوحة STM32 F103C8T6 التطويرية لا تحوي مبرمجة مدمجة مثل Arduino UNO ، لذا فهي بحاجة لدارة خارجية للبرمجة تُربط مع الحاسب وتدعى FTDI و تُربط مع اللوحة بالشكل التالي :



و يوجد نوع آخر يدعى ST-Link V2 :



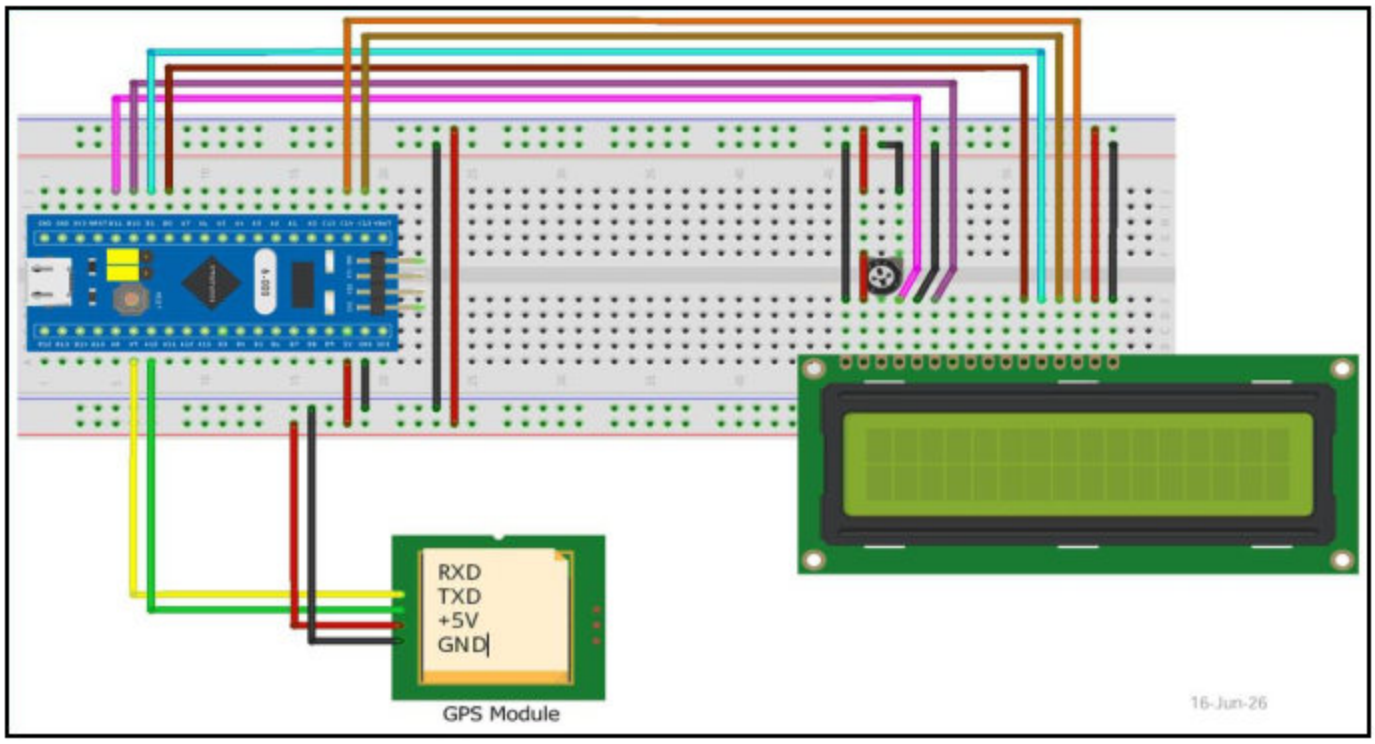
التطبيق الأول : الوميض Blinking

تطبيق Blink بسيط لتشغيل وإيقاف LED موصول على المنفذ 13 بمعدل 1000 ms :

```
void setup ()
{
  pinMode (PC13,OUTPUT);
}
void loop ()
{
  digitalWrite (PC13,HIGH);
  delay(1000);
  digitalWrite (PC13,LOW);
  delay(1000);
}
```



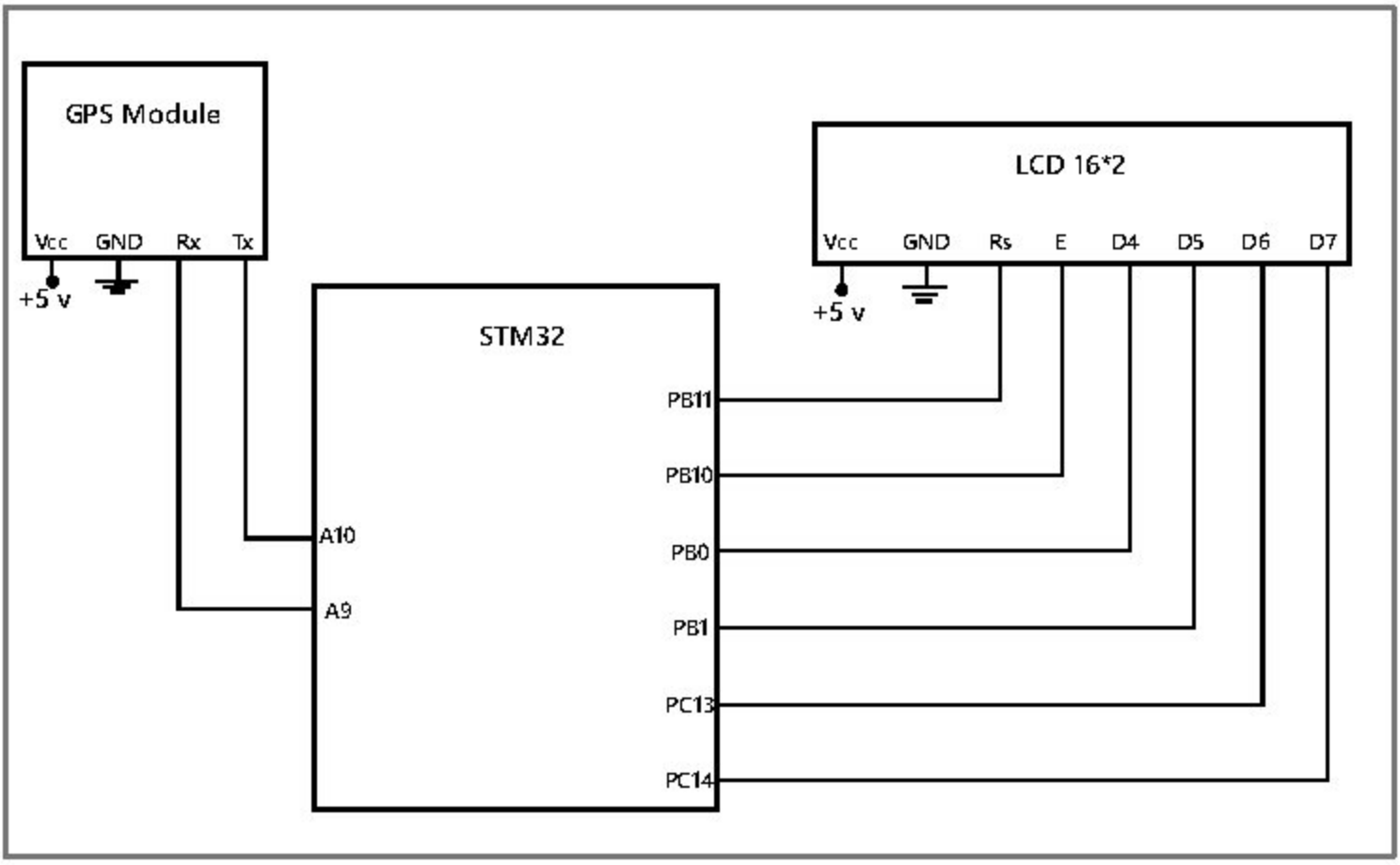
التطبيق الثاني : استخدام حساس GPS لتحديد الموقع



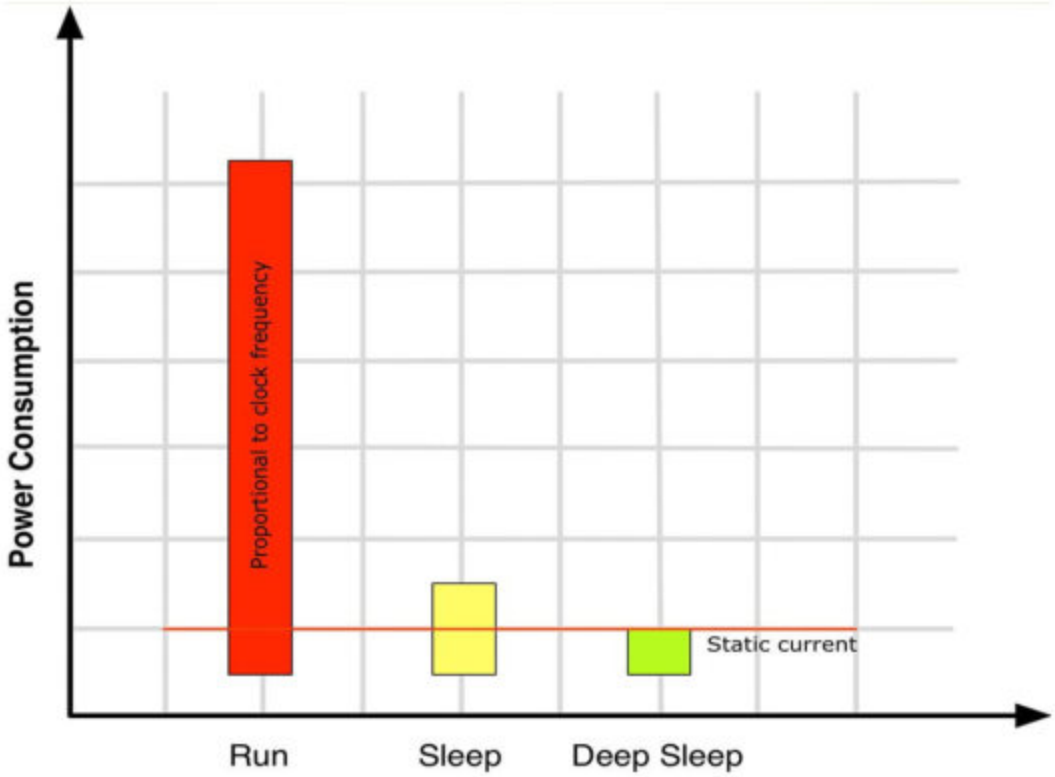

```
#include <LiquidCrystal.h>        // استدعاء مكتبة شاشة LCD
#include <TinyGPS++.h>            // استدعاء مكتبة GPS
const int rs = PB11, en = PB10, d4 = PB0, d5 = PB1, d6 = PC13, d7 = PC14;          // تعريف أرجل LCD
LiquidCrystal lcd(rs, en, d4, d5, d6, d7);        // إنشاء كائن للشاشة مع تحديد أطراف التوصيل مع الأردوينو
TinyGPSPlus gps;              // إنشاء كائن gps من الكلاس TinyGPSPlus لمعالجة بيانات GPS
void setup ( )
{
 Serial.begin(9600);              //  تشغيل الاتصال التسلسلي بسرعة 9600
 lcd.begin(16,2);                 // تهيئة شاشة LCD للعمل بنظام 16 عمود و 2 سطر
}
void loop ( )
{
 GPSDelay(1000);                  // استدعاء دالة تأخير خاصة لمدة 1 ثانية مع قراءة بيانات GPS أثناء الانتظار
 double lat_val, lng_val;         // تعريف متغيرين لحفظ خط العرض (Latitude) وخط الطول (Longitude)
 bool loc_valid;                  // متغير منطقي (True/False) لتحديد إذا كانت إشارة GPS صالحة أم لا
 loc_valid = gps.location.isValid();          // التحقق من أن موقع GPS صحيح وتم استقبال بيانات كافية
```

```
lat_val = gps.location.lat();                    // استخراج قيمة خط العرض من وحدة GPS
lng_val = gps.location.lng();                    // استخراج قيمة خط الطول من وحدة GPS
if (!loc_valid)                                  // إذا كانت البيانات غير صالحة (لا توجد إشارة GPS صحيحة)
{
  lcd.print("Waiting");                 // عرض كلمة Waiting على شاشة LCD لبيان انتظار إشارة GPS
  Serial.print("Latitude : ");          // طباعة عنوان خط العرض على Serial Monitor
  Serial.println("*");                  // طباعة نجوم بدل قيمة غير متوفرة
  Serial.print("Longitude : ");         // طباعة عنوان خط الطول على Serial Monitor
  Serial.println("*");                  // طباعة نجوم بدل قيمة غير متوفرة
  delay(4000);                          // انتظار 4 ثواني قبل إعادة المحاولة
  lcd.clear( );                         // مسح شاشة LCD لإعادة التحديث
}
else                    // في حال كانت بيانات GPS صحيحة ومتاحة
{
  lcd.clear( );                                  // مسح الشاشة قبل عرض بيانات جديدة
  Serial.println("GPS READING: ");               // طباعة رسالة بداية قراءة GPS
  Serial.print("Latitude : ");                   // طباعة عنوان خط العرض
  Serial.println(lat_val, 6);                    // طباعة قيمة خط العرض بدقة 6 أرقام عشرية على Serial Monitor
  lcd.setCursor(0,0);                            // وضع المؤشر في السطر الأول بداية الشاشة
  lcd.print("LAT:");                             // كتابة كلمة LAT على LCD
  lcd.print(lat_val, 6);                         // عرض قيمة خط العرض على الشاشة بدقة 6 أرقام عشرية
  Serial.print("Longitude : ");                  // طباعة عنوان خط الطول
  Serial.println(lng_val, 6);                    // طباعة قيمة خط الطول على Serial Monitor
  lcd.setCursor(0,1);                            // نقل المؤشر إلى السطر الثاني
  lcd.print("LONG:");                            // كتابة كلمة LONG على LCD
  lcd.print(lng_val, 6);                         // عرض قيمة خط الطول على الشاشة بدقة 6 أرقام عشرية
  delay(4000);                                   // انتظار 4 ثواني قبل تحديث القراءة مرة أخرى
}
```

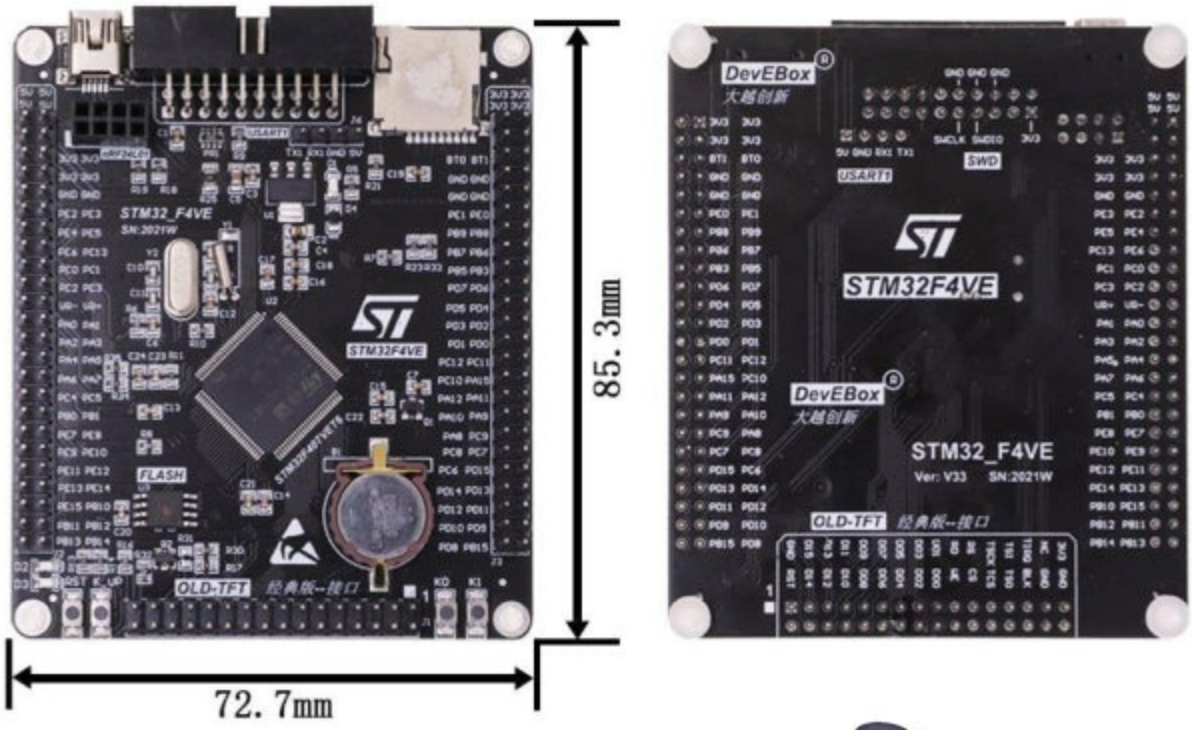
```
}
static void GPSDelay(unsigned long ms)                  // دالة تأخير خاصة مع تشغيل قراءة GPS أثناء الانتظار //
{
 unsigned long start = millis();                        // حفظ الزمن الحالي منذ تشغيل التحكم (بالملي ثانية) //
 do
 {
 while (Serial.available( ))                // Serial طالما يوجد بيانات قادمة من GPS عبر
   gps.encode(Serial.read( ));              // قراءة البيانات حرفاً حرفاً وتمريرها لمكتبة TinyGPS لتحليلها //
 }
 while (millis() - start < ms);             // الاستمرار حتى انتهاء مدة التأخير المطلوبة //
}
```



يوضح المخطط التالي استهلاك الطاقة في معالجات Cortex-M في أوضاع الطاقة المختلفة:



مثال آخر على لوحات التطوير STM32 F4VE:





انتهى بحث STM32..

ESP32 MICROCONTROLLER

- تم تصميم ESP32 للتطبيقات الخاصة بالإلكترونيات المحمولة، والأجهزة القابلة للارتداء، وتطبيقات إنترنت الأشياء (IoT).

- يتميز بجميع الخصائص المتطورة الخاصة بالشرائح منخفضة الاستهلاك للطاقة، بما في ذلك التحكم الدقيق بتفعيل وإيقاف إشارة الساعة (Fine-grained Clock Gating)، ووجود عدة أنماط لتوفير الطاقة، والتحكم الديناميكي بمستوى استهلاك الطاقة.

على سبيل المثال... في تطبيقات وحدة تجميع بيانات الحساسات منخفضة الطاقة الخاصة بإنترنت الأشياء ، يتم إيقاظ ESP32 بشكل دوري فقط عند اكتشاف شرط أو حالة محددة. ويتم استخدام دورة تشغيل منخفضة (Low-duty Cycle) لتقليل كمية الطاقة التي تستهلكها الشريحة.

كما أن خرج مضخم القدرة (Power Amplifier) قابل للضبط، مما يساهم في تحقيق توازن مثالي بين مدى الاتصال، ومعدل نقل البيانات، واستهلاك الطاقة.

- يُعد ESP32 حلاً متكاملاً بدرجة عالية لتطبيقات إنترنت الأشياء المعتمدة على Wi-Fi و Bluetooth.

حيث يدمج ESP32 العديد من المكونات ضمن شريحة واحدة، منها:

- ✓ دائرة تبديل الهوائي (Antenna Switch)
- ✓ دائرة موازنة الترددات الراديوية (RF Balun)
- ✓ مضخم القدرة (Power Amplifier)
- ✓ مضخم الاستقبال منخفض الضجيج (Low-Noise Receive Amplifier)
- ✓ المرشحات (Filters)
- ✓ وحدات إدارة الطاقة (Power Management Modules)

وبذلك يشغل الحل الكامل مساحة صغيرة جداً على لوحة الدارة المطبوعة (PCB).

———————————————————

الميزات الأساسية لشبكة Wi-Fi

1. يدعم ESP32 بروتوكول TCP/IP وبروتوكول التحكم بالوصول للوسائط الكامل الخاص بشبكات Wi-Fi وفق معيار 802.11 b/g/n.

2. يدعم معيار 802.11 n ضمن نطاق التردد 2.4 GHz، مع سرعة نقل تصل إلى 150 Mbps.

3. يدعم تقنية (Wi-Fi Multimedia) WMM الخاصة بتحسين إدارة الوسائط المتعددة عبر شبكة Wi-Fi.

الميزات الأساسية لـ Bluetooth

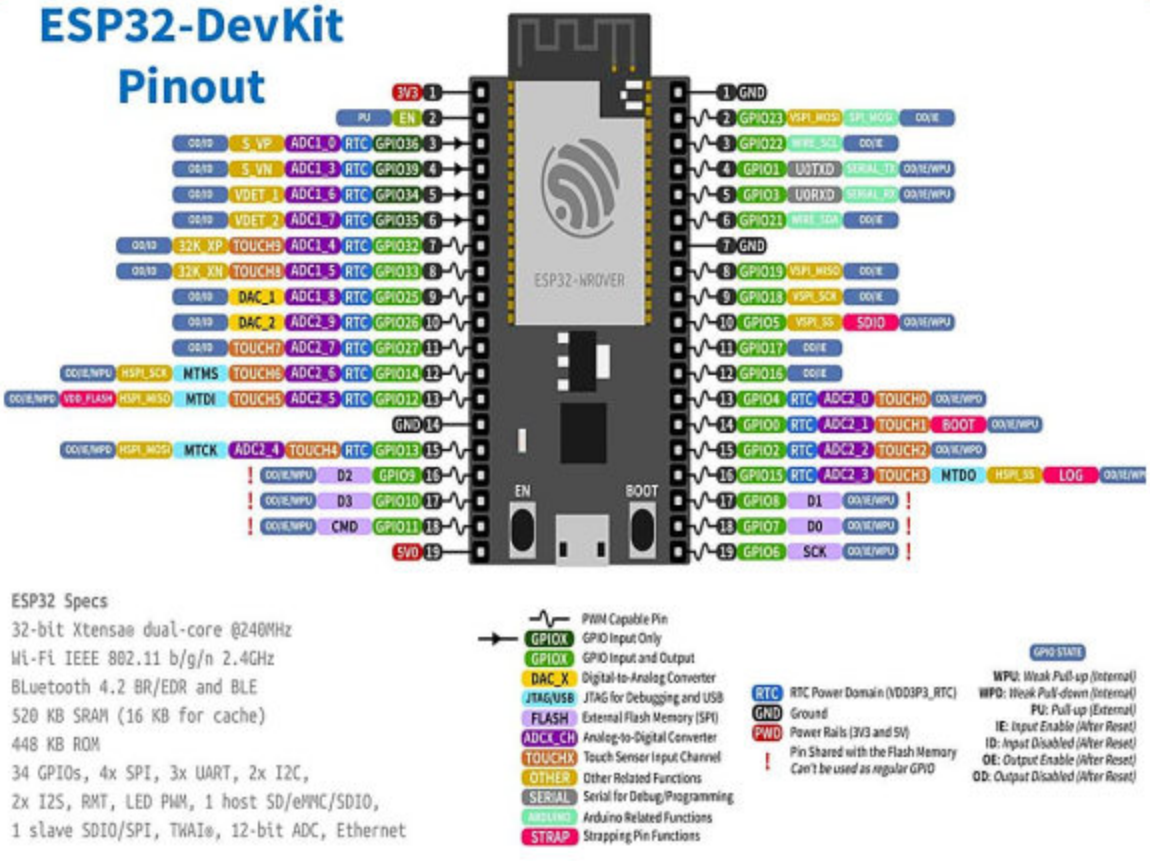
1. متوافق مع مواصفات Bluetooth v4.2 و Bluetooth Low Energy (BLE).

2. يدعم واجهة UART HCI عالية السرعة بمعدل نقل يصل إلى 4 Mbps.

3. يدعم الاتصالات المتعددة في كل من Bluetooth التقليدي (Classic Bluetooth) Bluetoothg منخفض الطاقة (Bluetooth LE).

4. يدعم العمل المتزامن بين البث التعريفي (Advertising) والبحث/المسح (Scanning).

الصورة التالية تبين أطراف المتحكم ESP32



بعض استخدامات المتحكم ESP32 :

- ✓ مركز مستشعرات إنترنت الأشياء منخفضة الطاقة (Generic Low-power IoT Sensor Hub)
- ✓ مسجلات بيانات إنترنت الأشياء منخفضة الطاقة (Generic Low-power IoT Data Loggers)
- ✓ كاميرات لبث الفيديو (Cameras for Video Streaming)
- ✓ أجهزة البث عبر الإنترنت (Over-the-top (OTT) Devices)
- ✓ التعرف على الكلام (Speech Recognition)
- ✓ التعرف على الصور (Image Recognition)
- ✓ شبكات Mesh (Mesh Network)



• الأتمتة المنزلية (Home Automation)

- التحكم بالإضاءة (Light control)
- المقابس الذكية (Smart plugs)
- أقفال الأبواب الذكية (Smart door locks)

• المباني الذكية (Smart Building)

- الإضاءة الذكية (Smart lighting)
- مراقبة الطاقة (Energy monitoring)

• الأتمتة الصناعية (Industrial Automation)

- التحكم اللاسلكي الصناعي (Industrial wireless control)
- الروبوتات الصناعية (Industrial robotics)

• الزراعة الذكية (Smart Agriculture)

- البيوت الزراعية الذكية / البيوت المحمية الذكية (Smart greenhouses)
- الري الذكي (Smart irrigation)
- الروبوتات الزراعية (Agriculture robotics)

- **تطبيقات الصوت (Audio Applications)**

- مشغلات الموسيقى عبر الإنترنت (Internet music players)
- أجهزة البث المباشر (Live streaming devices)
- مشغلات الراديو عبر الإنترنت (Internet radio players)
- سماعات الرأس الصوتية (Audio headsets)

- **تطبيقات الرعاية الصحية (Health Care Applications)**

- مراقبة الحالة الصحية (Health monitoring)
- أجهزة مراقبة الأطفال (Baby monitors)

- **الألعاب المزوّدة بتقنية Wi-Fi (Wi-Fi-enabled Toys)**

- ألعاب يتم التحكم بها عن بُعد (Remote control toys)
- ألعاب تعتمد على استشعار القرب (Proximity sensing toys)
- الألعاب التعليمية (Educational toys)

- **الإلكترونيات القابلة للارتداء (Wearable Electronics)**

- الساعات الذكية (Smart watches)
- الأساور الذكية (Smart bracelets)

- **تطبيقات البيع بالتجزئة وخدمات الطعام (Retail & Catering Applications)**

- أجهزة نقاط البيع (POS machines)
- روبوتات الخدمة (Service robots)

أنماط الطاقة في لوحة ESP32 - (ESP32 Power Mode) :

- **النمط النشط (Active Mode):**

تكون وحدة الاتصال اللاسلكي في الشريحة قيد التشغيل، حيث يستطيع المعالج استقبال البيانات أو إرسالها أو الاستماع إلى الإشارات.

- **نمط السكون للمودم (Modem-Sleep Mode):**

يبقى المعالج المركزي (CPU) قيد العمل، ويمكن ضبط تردد الساعة حسب الحاجة. بينما يتم تعطيل النطاق الأساسي (Baseband) الخاص بتقنيتي Wi-Fi و Bluetooth بالإضافة إلى وحدة الإرسال والاستقبال اللاسلكي (Radio).

◆ نمط السكون الخفيف (Light-Sleep Mode):

يتم إيقاف عمل المعالج المركزي مؤقتًا، بينما تبقى ذاكرة RTC ووحداتها الطرفية، بالإضافة إلى المعالج المساعد منخفض الطاقة ULP Coprocessor، قيد التشغيل.

يمكن لأي حدث من أحداث إيقاظ المعالج (مثل إشارات MAC أو المضيف Host أو مؤقت RTC أو المقاطعات الخارجية) أن يعيد تشغيل الشريحة من وضع السكون.

◆ نمط السكون العميق (Deep-Sleep Mode):

تبقى فقط ذاكرة RTC والوحدات الطرفية الخاصة بها قيد التشغيل. يتم حفظ معلومات اتصال Wi-Fi و Bluetooth داخل ذاكرة RTC، كما يبقى المعالج المساعد منخفض الطاقة ULP فعالًا.

◆ نمط السبات (Hibernation Mode):

يتم تعطيل المذبذب الداخلي بتردد 8 MHz والمعالج المساعد ULP، كما يتم إيقاف تشغيل ذاكرة الاستعادة الخاصة بـ RTC.

يبقى فعالًا فقط مؤقت واحد تابع لـ RTC يعمل بواسطة الساعة البطيئة، بالإضافة إلى بعض منافذ الإدخال/الإخراج الخاصة بـ RTC GPIO.

يمكن لمؤقت RTC أو منافذ RTC GPIO إيقاظ الشريحة وإخراجها من وضع السبات.

المعالج و الذاكرة

◆ المعالج الدقيق (Microprocessor):

يحتوي ESP32 على معالج دقيق من نوع Xtensa® LX6 بمعمارية 32 بت، ويتوفر بإصدار أحادي النواة أو ثنائي النواة (Single-core / Dual-core)، ويعمل بتردد يصل إلى 240 MHz.

◆ ذاكرة القراءة فقط (ROM):

تحتوي الشريحة على ذاكرة ROM بسعة 448 KB.

◆ ذاكرة الوصول العشوائي الساكنة (SRAM):

تحتوي على ذاكرة SRAM بسعة 520 KB.

◆ ذاكرة SRAM الخاصة بـ RTC:

توجد ذاكرة SRAM إضافية تابعة لوحدة RTC بسعة 16 KB.

Clock and Timer

- **المذبذب الداخلي بتردد 8 MHz (Internal 8 MHz Oscillator):**

يحتوي ESP32 على مذبذب داخلي يعمل بتردد 8 MHz، مع إمكانية إجراء المعايرة (Calibration) لتحسين دقة التردد واستقرار التوقيت.

- **مذبذب RC الداخلي (Internal RC Oscillator):**

تحتوي الشريحة على مذبذب داخلي من نوع RC Oscillator، ويمكن معايرته أيضًا لضبط دقة التردد المستخدم في عمليات التوقيت وتشغيل بعض وظائف النظام.

- **المذبذب البلوري الخارجي (External Crystal Oscillator):**

يدعم ESP32 مذبذبًا بلوريًا خارجيًا بتردد يتراوح بين 2 MHz و60 MHz، ويُستخدم تردد 40 MHz بشكل أساسي لتشغيل وحدات الاتصال اللاسلكي Wi-Fi و Bluetooth.

- **المذبذب البلوري الخارجي بتردد 32 kHz لوحدة RTC:**

يدعم مذبذبًا بلوريًا خارجيًا بتردد 32 kHz خاصًا بوحدة RTC (Real-Time Clock)، مع إمكانية معايرته للحصول على توقيت أكثر دقة.

مجموعتا المؤقتات (Two Timer Groups):

يحتوي ESP32 على مجموعتين من المؤقتات :

- مؤقت RTC (RTC Timer)
- مؤقت مراقبة RTC (RTC Watchdog)



وتتضمن كل مجموعة :

- ✓ مؤقتين بسعة 64 بت (2 x 64-bit Timers) لاستخدامهما في عمليات القياس الزمني والتحكم بتوقيت تنفيذ المهام.
- ✓ مؤقت مراقبة رئيسي Main Watchdog Timer لمراقبة عمل المعالج وإعادة تشغيل النظام تلقائيًا عند حدوث توقف أو خلل.

واجهات ووحدات الطرفيات المتقدمة (Advanced Peripheral Interfaces):

منافذ الإدخال والإخراج العامة القابلة للبرمجة (Programmable GPIOs):

يحتوي ESP32 على 34 منفذ GPIO قابلة للبرمجة.

المحول التناظري إلى الرقمي (ADC):

يحتوي على محول ADC بدقة 12 بت، ويدعم حتى 18 قناة، ويُستخدم لتحويل الإشارات التناظرية القادمة من الحساسات إلى قيم رقمية يمكن للمعالج التعامل معها.

المحول الرقمي إلى التناظري (DAC):

يحتوي على محولي DAC بدقة 8 بت، ويُستخدمان لتحويل الإشارات الرقمية إلى إشارات تناظرية مثل توليد الجهود أو الإشارات الصوتية.

حساسات اللمس (Touch Sensors):

يدعم 10 حساسات لمس يمكن استخدامها لاكتشاف اللمس دون الحاجة إلى أزرار ميكانيكية.

واجهة SPI:

يحتوي على 4 وحدات SPI.

واجهة I2S

يدعم واجهتي I2S، وتُستخدم بشكل أساسي لنقل البيانات الصوتية الرقمية بين المعالج والأجهزة الصوتية مثل المحولات الصوتية.

واجهة I2C

يحتوي على واجهتي I2C، وهي بروتوكولات اتصال تسلسلية تُستخدم لربط عدة أجهزة طرفية مثل الحساسات ودوائر التحكم باستخدام خطين فقط.

واجهة UART:

يحتوي على 3 وحدات UART للاتصال التسلسلي، وتُستخدم للتواصل مع أجهزة مثل وحدات GPS، الحواسيب، ووحدات الاتصال الخارجية.

واجهة CAN:

يدعم بروتوكول CAN، ويُستخدم في تطبيقات التحكم الصناعي والسيارات التي تتطلب اتصالًا موثوقًا بين الوحدات.

- يدعم توليد إشارات *PWM*.

- حساس المجال المغناطيسي (*Hall Sensor*):

يحتوي ESP32 على حساس Hall Sensor مدمج يمكنه قياس شدة المجال المغناطيسي، ويُستخدم في تطبيقات الكشف عن القرب أو قياس الحركة.

الجدول التالي يحوي مقارنة بين المتحكمات التي تعاملنا معها :

مقارنة بين لوحات Arduino و STM32F103C و ESP32			
ESP32	STM32F103C (Blue Pill)	Arduino UNO	المواصفة
			
Xtensa Dual-Core LX6 (ESP32)	ARM Cortex-M3 (STM32F103C8T6)	ATmega328P	Microcontroller
32bit	32bit	8bit	Architecture
4MB	64 KB	32 KB	Flash Memory
520 KB	20 KB	2 KB	SRAM
240MHZ/160MHZ	72 MHz	16 MHz	Clock Speed
36GPIO	37 GPIO	14 Digital I/O	Digital Pins (GPIO)
18 قناة 12-bit	10 قنوات 12-bit	6 قنوات 10-bit	ADC Pins
2 DAC (8-bit)	2 DAC (8-bit)	لا يوجد	DAC Pins
3	3	1	عدد واجهات UART
4	2	1	عدد واجهات SPI
2	2	1	عدد واجهات I²C
1	1	لا يوجد داخلياً	عدد واجهات CAN
2	2	لا يوجد	عدد واجهات I²S
16	12	6	PWM Pins
✅ موجود (802.11 b/g/n)	❌ لا يوجد	❌ لا يوجد	WiFi
✅ موجود (Bluetooth Classic + BLE)	❌ لا يوجد	❌ لا يوجد	Bluetooth

تطبيقات على المتحكم ESP32

التطبيق الأول :

يقوم هذا البرنامج بربط لوحة ESP32 بشبكة WiFi ثم يعرض عنوان الـ IP الذي حصلت عليه اللوحة على شاشة Serial monitor

```
#include <WiFi.h>                                   // تضمين مكتبة Wi-Fi الخاصة بلوحة ESP32
const char* ssid = "Wifi_Name";                     // اسم شبكة Wi-Fi المراد الاتصال بها
const char* password = "12345678";                  // كلمة مرور شبكة Wi-Fi
void setup( )
{
 Serial.begin(115200);                              // تهيئة الاتصال التسلسلي بسرعة 115200 بت/ثانية
 WiFi.begin(ssid, password);                        // بدء الاتصال بالشبكة باستخدام الاسم وكلمة المرور
 Serial.println("Connecting");                      // طباعة رسالة تفيد ببدء محاولة الاتصال
 while (WiFi.status( ) != WL_CONNECTED)             // الاستمرار في التحقق حتى يتم الاتصال بالشبكة
 {
  delay(500);                                       // الانتظار لمدة 500 ميلي ثانية بين كل محاولة
  Serial.print(".");                                // طباعة نقطة للدلالة على استمرار محاولة الاتصال
 }
 Serial.println(" ");                               // الانتقال إلى سطر جديد بعد نجاح الاتصال
 Serial.print("Connected to WiFi network with IP Address: ");          // طباعة رسالة نجاح الاتصال
 Serial.println(WiFi.localIP( ));                                      // طباعة عنوان IP الذي حصلت عليه اللوحة
}
void loop( )
{
            // لا يوجد أي كود هنا
}
```

ملاحظة :

نلاحظ أنه عند تعريف المتغير ssid تم وضع علامة * بعد كلمة char وهذه العلامة هنا تدل على أن المتغير ssid يخزن عنوان أول مِحرف في اسم الشبكة (WiFi-Name) أي عنوان حرف w ، ولا يخزن اسم الشبكة ، الأمر نفسه ينطبق على المتغير Password .

لكن لمَ لم نستعمل متغير من نوع string ؟

لأن مكتبة WiFi صممت للتعامل مع متغيرات من نوع const char *

التطبيق الثاني :

يقوم هذا البرنامج بطباعة إحداثيات الموقع على شاشة Serial monitor ، ولكن دون توصيل وحدة GPS مع لوحة ESP32 و إنما سيتم ربط لوحة ESP32 مع تطبيق Dabble على الهاتف ، والذي يتيح الوصول لحساسات الهاتف الداخلية مثل GPS و Gyroscope و microphone وغيرها .. و إرسالها إلى لوحة ESP32

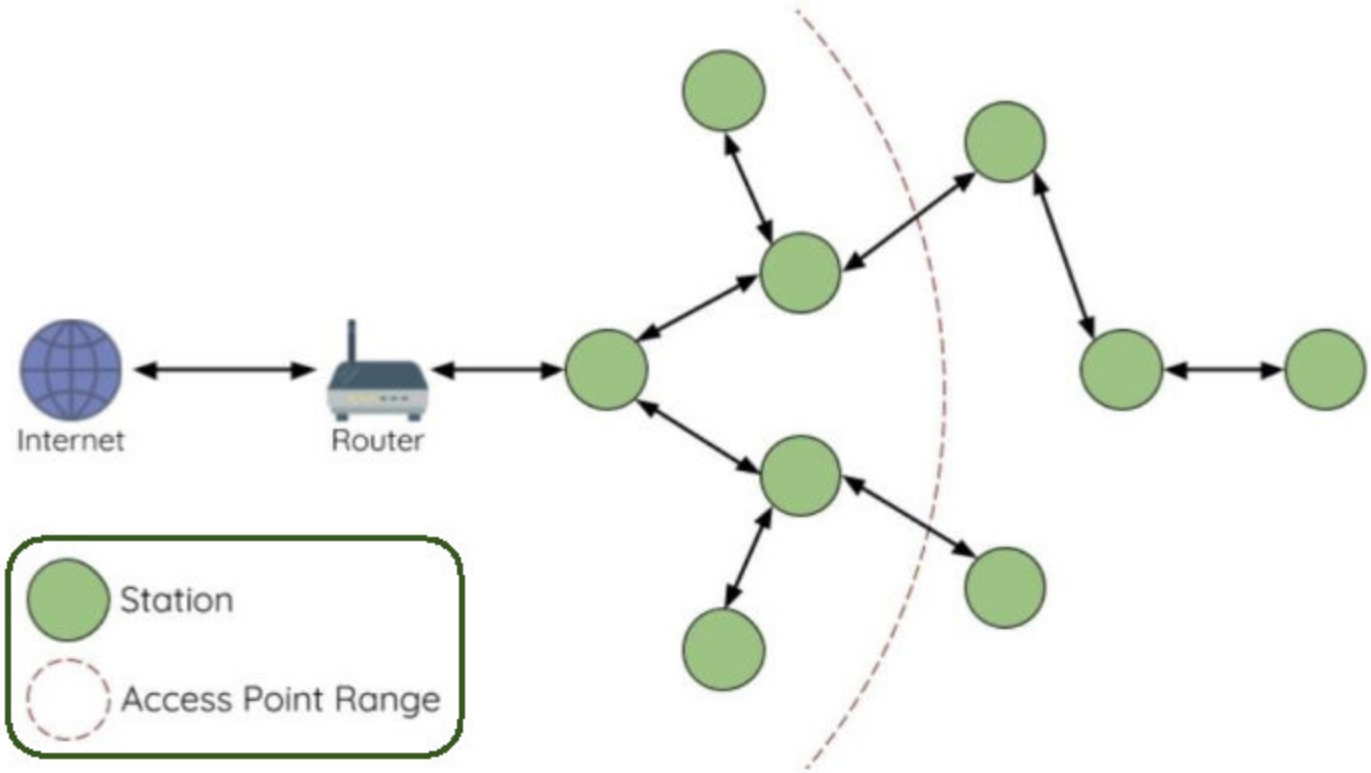
أي أن تطبيق Dabble سيجلب بيانات الموقع من الـ GPS الخاص بالهاتف و يرسلها إلى لوحة ESP32 لعرضها على Serial monitor

```
#define CUSTOM_SETTINGS                 // تفعيل إمكانية اختيار الوحدات المطلوبة من مكتبة Dabble
#define INCLUDE_SENSOR_MODULE           // تضمين وحدة الحساسات Sensor Module من تطبيق Dabble
#include <DabbleESP32.h>                // تضمين مكتبة Dabble الخاصة بالاتصال بين ESP32 والهاتف
void setup( )
{
 Serial.begin(115200);                  // تهيئة الاتصال التسلسلي بسرعة 115200 بت/ثانية
 Dabble.begin("MyEsp32");               // بدء اتصال ESP32 مع الهاتف باسم MyEsp32
}
void loop( )
{
 Dabble.processInput( );                // تحديث البيانات المستلمة من تطبيق Dabble
 print_GPS_data( );                     // استدعاء الدالة الخاصة بطباعة بيانات GPS
 delay(1000);                           // تأخير لمدة ثانية واحدة بين كل قراءة
}
```

```
void print_GPS_data( )                          // تعريف دالة لطباعة بيانات الموقع الجغرافي
{
 Serial.print("Longitude: ");                   // طباعة كلمة Longitude للدلالة على خط الطول
 Serial.print(Sensor.getGPSlongitude( ), 7);    // قراءة وطباعة خط الطول من GPS في Dabble بدقة 7 منازل عشرية
   وحدة
 Serial.print( ' \t ' );                        // طباعة مسافة Tab للفصل بين خط الطول وخط العرض
 Serial.print("Latitude: ");                    // طباعة كلمة Latitude للدلالة على خط العرض
 Serial.println(Sensor.getGPSLatitude( ), 7);   // قراءة وطباعة خط العرض بدقة 7 منازل عشرية
 Serial.println( );                             // الانتقال إلى سطر جديد
}
```

شبكة Mesh :

إن المدى الأقصى لشبكة الـ WiFi هو 100m ، لكن إن أردت مثلاً ربط عدة أجهزة في منشأة صناعية كبيرة عندها نقوم بربطها عبر شبكة Mesh ، وهي نوع من شبكات الاتصال يتم فيها ربط عدة أجهزة مع بعضها بحيث لا يعتمد كل جهاز على نقطة اتصال واحدة فقط ، بل يمكن للأجهزة أن تتواصل مع بعضها مباشرة بالتالي يمكن تغطية مساحات أكبر.





تم بعون الله الانتهاء من بحث المتحكمات STM32 & ESP32 ...

اكتب ملاحظاتك ♡:

فريقنا يتمنى لكم دراسة موفقة ❤